第四章 项目启动：基于Vibe Coding搭建最小可行技能

> **本章导言**
>
> 在前三章中，我们完成了从概念建立、方法拆解、AI能力映射到Skill架构设计与评估框架的全部准备工作。你手中应该已经握有一份完整的Skill设计文档、一张处理流程架构图、一套评估方案，以及一个初步验证交互逻辑的Gradio原型。然而，所有这些仍然停留在“设计”层面——它们是蓝图，而非建筑。
>
> 本章将带领你跨越从“设计”到“实现”的最后一道门槛。我们将以“可用性测试高亮时刻提取器”为示范案例，完整演示一个Vibe Coding驱动的开发流程：从初始提示词到可运行脚本，从报错调试到功能迭代，从数据处理到AI分析集成。在此基础上，你将进入小组工作坊，完成自己Skill的数据准备、核心处理逻辑与基本交互流，并在MVP内部检视中展示第一个可运行的端到端版本。
>
> 学完本章，你将不再只是“会写提示词”的初学者——你将具备独立组织Vibe Coding开发会话、应对常见技术障碍、并在有限时间内交付最小可行产品的实战能力。

本章学习目标

完成本章学习后，读者应能够：

1. 按照“分轮推进”的原则，用自然语言分阶段描述Skill的开发需求，并生成可运行的Python代码；
2. 为Skill准备合适的开发数据（真实数据脱敏或AI生成模拟数据），并编写数据预处理模块；
3. 设计有效的分析提示词，调用大模型API完成核心分析任务，并处理输出不稳定性；
4. 搭建基本的交互流（终端或Gradio），实现数据输入、处理、输出的闭环；
5. 记录Vibe Coding对话日志，并在MVP检视中进行成果展示与问题求助。

4.1 MVP开发的核心原则与整体路径

4.1.1 什么是最小可行技能（MVP Skill）？

在软件工程中，“最小可行产品”（Minimum Viable Product, MVP）指具备核心功能、能够交付用户价值的最小版本。对于本课程而言，一个MVP Skill应满足以下三个条件：

1. **可运行**：代码能够在开发环境中成功执行，不依赖开发者本人的特殊操作才能运行。
2. **端到端**：能够接收输入、执行核心AI处理、产生输出，形成一个完整的信息流闭环。
3. **有意义**：输出不是占位符或测试垃圾，而是与方法目标相关的分析结果（即使准确率不高）。

一个MVP Skill不要求：

- 功能完备（可以只实现设计文档中的一个核心环节）
- 界面精美（终端输出或极简Gradio即可）
- 高准确率（准确率优化是第5周的迭代任务）
- 处理所有边界情况（可以约定输入数据的格式和范围）

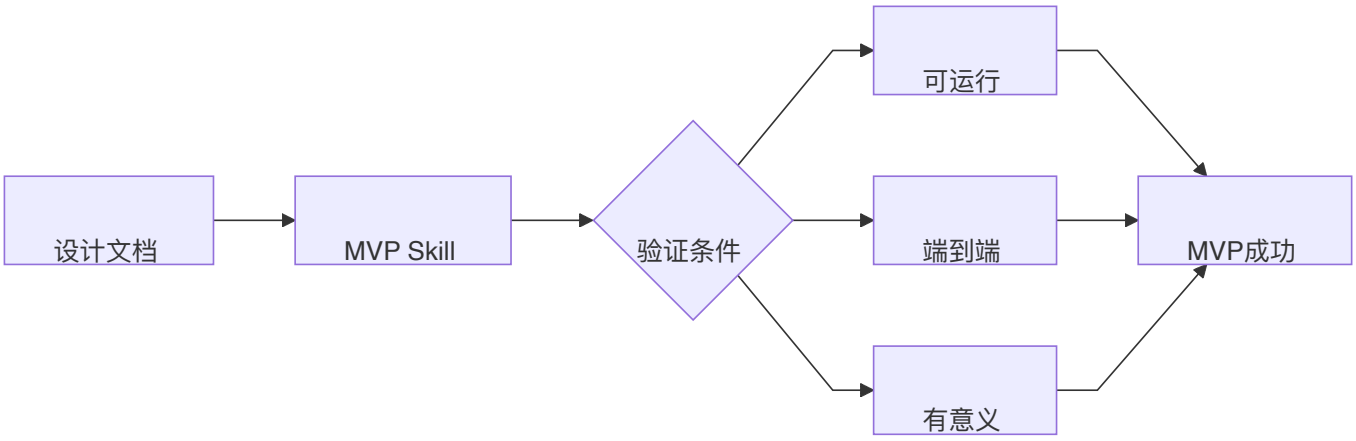


图4-1 MVP Skill验证条件

4.1.2 分轮推进：Vibe Coding开发的黄金法则

在第一章和第二章中，我们反复强调一个原则：**一次只让AI做一件事**。在MVP开发阶段，这一原则尤其重要。如果你试图用一个提示词让AI生成整个Skill，你会得到一个数百行、难以调试的"代码团"——它可能看起来完整，但一旦报错，你几乎无法定位问题出在数据读取、预处理、API调用还是输出格式化。

因此，我们遵循**三轮推进**的策略：

轮次	目标	典型代码产出	可检验的标准
第一轮	数据准备与预处理	数据加载、清洗、分段、格式化函数	预处理后的数据能正确打印出来
第二轮	核心AI分析	调用LLM API的分析函数、提示词设计、输出解析	对3-5条测试数据能产出结构化结果
第三轮	交互流与集成	终端或Gradio入口、将前两轮串联起来的"胶水代码"	外部用户能按指引输入并获得输出

每一轮的目标都是产生一个**可独立检验的成果**。第一轮结束时，你应该能运行预处理模块，看到清洗后规整的数据。第二轮结束时，你应该能用分析函数处理单条数据，看到返回的字典。第三轮结束时，你应该能运行整个流程，看到从原始输入到最终输出的完整轨迹。

这种"分步推进、逐层检验"的方式，在Vibe Coding中不仅是效率策略，更是认知策略——它逼迫你在每一步都清晰地知道"当前AI做的这件事的输入是什么、输出应该是什么"。当你能够精确地回答这两个问题时，你与AI的对话质量将大幅提升。

4.1.3 本章指南的使用方式

本章的后续内容将按照"示范-指南-实践"的结构组织：

- **第4.2节**：导师示范开发一个完整的MVP Skill（可用性测试高亮时刻提取器），展示三轮推进的全过程。
- **第4.3节**：数据准备指南，涵盖真实数据脱敏、AI生成模拟数据、数据预处理Vibe Coding。
- **第4.4节**：核心分析逻辑开发指南，包括分析提示词设计原则、输出稳定性处理、常见问题应对。

- **第4.5节**：交互流实现指南，终端交互与Gradio界面的Vibe Coding描述模板。
- **第4.6节**：MVP内部检视，展示标准、互助流程与跨组学习。

无论你选择的方法是什么，本章提供的提示词模板、调试策略和迭代习惯都是通用的。建议在阅读示范部分时，同步思考"如果我的方法是X，对应的第一轮提示词应该怎么写"。

4.2 示范：可用性测试高亮时刻提取器的完整Vibe Coding开发

本节以"可用性测试高亮时刻提取器"为案例，完整演示从零到可运行MVP的三轮Vibe Coding开发。这个案例的选择具有代表性——它涉及非文本数据（视频帧）、多模态AI分析、结构化输出生成，这些技术要素在访谈分析（语音转文字+文本分析）、亲和图（文本聚类）等常见Skill中也会以不同形式出现。

4.2.1 项目定义与范围约束

项目名称：Usability Highlight Extractor

核心功能：用户上传一段可用性测试录屏视频（MP4），Skill自动按每秒1帧提取关键帧，调用多模态大模型分析每一帧，识别其中可能存在可用性问题的时刻（困惑、错误、卡顿），生成高亮时刻列表和Markdown报告。

MVP范围约束（刻意控制复杂度）：

- 只处理前20帧（而非完整视频的所有帧）
- 只检测三类信号：用户面部表情（皱眉/困惑）、鼠标行为（反复点击/长时间静止）、界面异常（错误弹窗/空白页）
- 输出为终端打印的摘要 + JSON文件 + Markdown报告
- 使用DeepSeek API（如不支持视觉则fallback到模拟输出）

4.2.2 第一轮：搭建项目骨架与视频帧提取

初始提示词：

```
我想做一个"可用性测试高亮时刻提取器"。用户上传一段可用性测试的录屏视频（mp4），系统自动找到视频中用户可能遇到问题的时刻，生成一个高亮时刻列表。

请帮我搭建这个项目的基础代码框架：

1. 使用OpenCV读取视频文件，按照每秒1帧的间隔提取关键帧（避免每帧都处理，减少计算量）
2. 将提取的帧保存到临时文件夹，文件名为帧的时间戳（如 00_01_15.jpg 表示1分15秒）
3. 在提取过程中显示进度条

代码结构要求：
- 将视频处理相关功能放在一个独立的 `VideoProcessor` 类中
- 使用配置文件或常量来管理参数（如提取帧间隔、临时文件夹路径）
- 添加充分的中文注释
- 包含异常处理（文件不存在、视频无法打开等）

需要安装的Python库请一并告诉我。
```

提示词设计解析：

- **先定义产品愿景**：第一句话让AI理解整体目标，为后续细节指令提供上下文。
- **限定本次请求范围**：明确要求“基础代码框架”，而不是完整的分析工具。
- **指定技术栈和逻辑**：给出OpenCV、每秒1帧、保存文件名格式等具体规格，避免AI做出不合理的技术选择。
- **架构约束**：要求使用独立类、配置文件、注释，确保生成代码的可维护性。
- **依赖列表**：让AI在回复中一并告知需要安装的库，减少后续查找时间。

AI生成后的处理流程：

1. 浏览生成的代码，确认是否包含`VideoProcessor`类、`extract_frames`方法、进度条逻辑。
2. 安装依赖：`pip install opencv-python tqdm`。
3. 准备一段3分钟的测试视频（或下载公开的可用性测试示例视频）。
4. 运行代码，观察临时文件夹中是否生成按时间戳命名的帧图片。
5. 如果遇到报错（如文件路径错误、OpenCV版本兼容问题），将报错信息完整粘贴给AI，请求修复。

第一轮产出：一个能够读取视频、提取帧、保存图片的Python脚本。此时还没有任何“智能”分析，但数据输入通路已经打通。

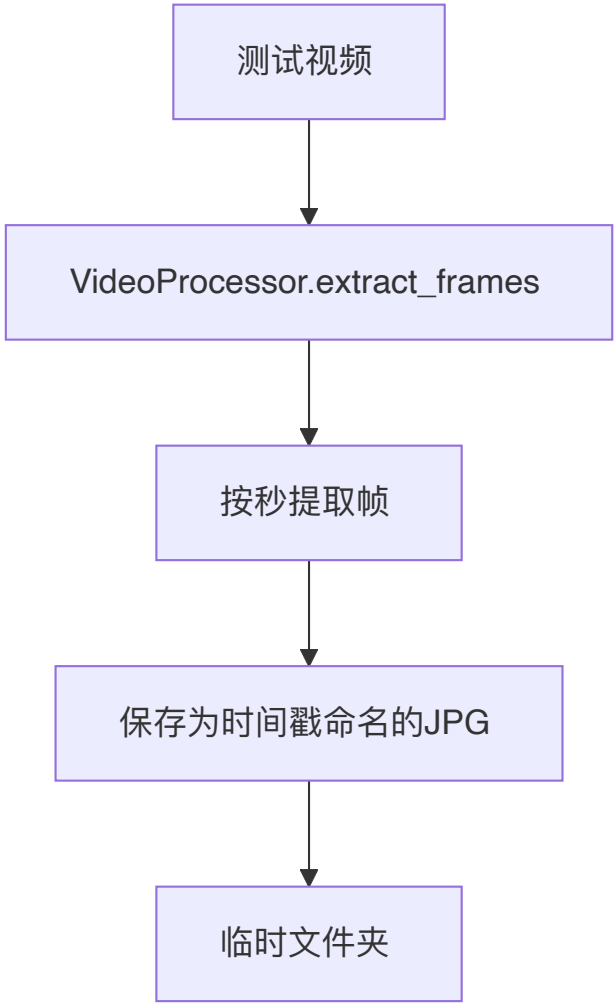


图4-2 第一轮架构：视频帧提取模块

4.2.3 第二轮：集成多模态分析

第二轮提示词：

```
很好，骨架代码跑通了。现在请在现有代码基础上增加核心分析功能：

1. 新增一个 `FrameAnalyzer` 类，负责调用多模态大模型的API。如果DeepSeek支持视觉，使用DeepSeek；如果不支持，使用OpenAI GPT-4V或Claude 3.5 Sonnet作为替代（请在注释中说明切换方式）。

2. `FrameAnalyzer` 需要有方法 `analyze_frame(image_path)`，接收帧图片路径，返回一个字典，包含：
   - `has_issue` (bool)：是否检测到可用性问题
   - `issue_type` (str)：问题类型，如"困惑/错误/卡顿/正常"
   - `description` (str)：问题的简短中文描述（20字以内）
   - `severity` (int)：严重度（1-5分）
   - `confidence` (float)：AI对自己判断的置信度（0-1）

3. 在分析提示词中，指导模型关注以下信号：
   - 用户面部表情（皱眉、困惑、沮丧）
   - 鼠标行为（反复点击、大范围晃动、长时间静止）
   - 界面状态（弹窗、错误提示、空白页面、加载中）

4. 在主流程中增加一个分析循环：遍历所有提取的帧 → 对每帧调用analyze_frame → 收集所有检测到问题的帧 → 按照视频时间戳排序 → 保存为JSON文件 `highlights.json`

5. API调用要求：使用指数退避重试（遇到限流等2秒、4秒、8秒），API Key从环境变量读取，每处理10帧输出一次进度。

6. 请保持之前要求的代码结构——VideoProcessor和FrameAnalyzer各自独立，主流程清晰串联两者。
```

提示词的关键设计：

- **容错性**：明确给出主方案（DeepSeek）和fallback方案（GPT-4V/Claude），避免因单个API不支持而卡住。
- **输出契约**：细致定义了返回字典的每个字段及其类型、取值范围，确保下游可以稳定处理。
- **领域知识注入**：将"面部表情、鼠标行为、界面状态"三类信号明确写入提示词，这是设计研究者的专业判断。
- **非功能需求**：重试策略、环境变量、进度输出，提升代码的健壮性。

第二轮运行中可能遇到的问题及解决：

问题	Vibe Coding解决方案
API调用返回格式不正确（带解释文字而非纯JSON）	修改提示词："请再次强调输出必须是纯净的JSON，不要添加任何解释。并在解析代码中增加容错：先尝试提取`json...`块，再尝试修复常见格式错误。"
处理20帧耗时过长，API费用超出预期	修改策略："请将分析循环改为只处理前10帧作为MVP验证。在代码中添加`max_frames`参数，默认为10，用户可通过命令行参数覆盖。"
某帧分析时API超时	重试机制应自动生效。如果持续超时，增加超时时间："请将API请求的超时从默认30秒增加到60秒。"

第二轮产出：一个能够逐帧调用多模态模型、收集问题帧、生成highlights.json的分析脚本。核心智能环节已经跑通。

4.2.4 第三轮：生成报告与交互流

第三轮提示词：

```
分析结果JSON已经生成。请在现有代码基础上增加报告生成功能：

1. 新增 `ReportGenerator` 类，读取 `highlights.json`，生成一份Markdown格式的分析报告
2. 报告内容：
   - 概览：视频总时长、分析帧数、检测到问题的帧数、问题帧占比
   - 问题时间线：按时间顺序列出所有问题时刻，每条包含时间戳、问题类型（用emoji表示）、描述、严重度（用星级表示）
   - 严重度分布：统计1-5分各段位的帧数
   - 建议优先关注：严重度≥4的时刻单独列出
   - 帧图片引用：每条问题时刻附上对应帧图片的文件路径（Markdown相对路径引用）
3. 报告保存为 `highlights_report.md`
4. 在主流程末尾调用ReportGenerator
5. 在最终输出中打印一个摘要：处理总帧数、检测到问题的帧数、报告保存路径

另外，请为整个工具增加一个简单的命令行交互入口。用户运行脚本时：
- 如果没有提供命令行参数，提示输入视频文件路径
- 提供参数时直接处理该文件
- 处理过程中显示清晰的分阶段进度（"正在提取帧..."、"正在分析帧..."、"正在生成报告..."）
```

第三轮的关键点：

- **报告结构设计**：不是随意输出，而是按照设计研究中常见的洞察报告格式组织（概览、时间线、分布、优先级）。这体现了Skill的"设计研究"属性。
- **可读性增强**：使用emoji、星级等视觉元素，让非技术利益相关者也能快速理解。
- **交互入口**：提供命令行参数和交互式输入两种方式，降低使用门槛。

最终运行演示：运行脚本，输入测试视频路径，观察终端输出进度，检查生成的highlights_report.md内容。

三轮开发的总结对比：

维度	第一轮	第二轮	第三轮
核心功能	视频帧提取	多模态分析	报告生成 + 交互
AI调用	无	有（LLM视觉API）	无（纯文本处理）
产出形态	图片文件	JSON数据	Markdown报告
可检验性	查看临时文件夹	查看JSON内容	打开报告文件

4.2.5 从示范中提炼的七个Vibe Coding习惯

在示范过程中，我们有意遵循了以下七个习惯。这些习惯是Vibe Coding高效开发的实践总结，应贯穿你所有开发会话。

习惯一：分轮推进，每轮只做一件事。每一轮提示词聚焦于一个层面的目标（数据流、分析、输出），产出可独立检验的中间结果。

习惯二：始终从"能跑通的最简版本"开始。第一轮甚至不包含任何AI调用，只验证数据的读取和转换。这看似缓慢，却能避免在复杂系统中迷失。

习惯三：在提示词中明确"输入输出契约"。每个函数都明确定义其参数类型、返回值结构，确保模块间能够无缝对接。

习惯四：把领域知识注入到提示词中。告诉AI应该关注哪些具体信号（如面部表情、鼠标行为），而不是笼统的"分析可用性问题"。

习惯五：把非功能需求写进提示词。重试逻辑、进度输出、环境变量管理、错误处理——这些"看不见"的质量特征需要显式要求。

习惯六：让报错信息代替你思考。遇到错误的第一反应不是自己分析，而是将完整的错误栈粘贴给AI，让AI帮助诊断。

习惯七：记录提示词与AI的回应。保留开发过程中的关键对话，作为后续调试、迭代和文档撰写的依据。

4.3 数据准备：真实数据、模拟数据与预处理

4.3.1 开发数据的三个来源

在开始编写核心分析代码之前，你必须确保有一批可用于开发和测试的数据。数据的质量和代表性直接影响你判断Skill是否"跑通"的能力。以下是三个来源及其适用场景。

来源一：真实数据（优先）

如果你在之前的课程项目、实习或自主研究中积累了与方法相关的真实数据（访谈录音/文字稿、问卷回复、用户反馈文本、可用性测试录像等），这是最理想的开发数据。真实数据包含了你在模拟中难以复制的"噪声"——口语中的犹豫和重复、问卷中的拼写错误、用户反馈中的不完整句子。这些正是你的Skill在实际使用中必须处理的，因此在开发阶段就暴露出来，远比在交付前才发现要好。

使用真实数据的注意事项：

- **合规性**：确保数据使用不违反保密协议、知情同意中的范围限制。如果是涉及个人身份信息的数据，必须进

行脱敏处理。

- **脱敏方法**：使用AI辅助脱敏。提示词示例："请写一个Python脚本，读取访谈文字稿，自动识别并替换其中的人名、地名、公司名、电话号码和邮箱地址，替换为【用户A】【某地】【某公司】等占位符。保留脱敏映射表。"
- **规模控制**：MVP阶段不需要完整数据集，抽取15-30条代表性样本即可。

来源二：AI生成的模拟数据（高效且可控）

如果你没有合适的真实数据，或者真实数据量不足，使用大模型生成模拟数据是完全可接受的，甚至在某些方面优于真实数据——你可以精确控制数据中各类样本的比例，确保测试覆盖到你关心的各种边界情况。

以下是为不同方法生成模拟数据的提示词模板：

为访谈分析生成模拟数据：

> 请为我的访谈分析Skill生成15条模拟用户访谈段落。背景：一款在线协作白板工具的用户研究。请生成不同情感倾向（5条正面、5条负面、5条中性）的访谈回答，涵盖不同话题（协作功能、界面易用性、性能速度、价格感知、与其他工具的集成）。每条80-150字。请保持语言自然，包含口语化表达、个别犹豫和重复，模拟真实访谈。

为亲和图生成模拟数据：

> 请为我的亲和图Skill生成30条研究数据点（便利贴条目）。背景：用户对一款移动支付App的使用反馈。条目内容应覆盖支付体验、账户安全、界面导航、客服体验、功能需求等不同方面。每条10-25字，用用户的语言风格表达。确保有5-8组语义相近的条目（模拟真实亲和图中可以被聚集到一起的条目）。

使用AI生成模拟数据后的必要步骤：
通读一遍生成的数据。AI有时会生成"模板化"内容——多条数据使用相似句式、表述过于工整。你需要删掉或修改这些明显不自然的内容。这只需要5-10分钟，但能显著提高测试数据的质量。

来源三：公开数据集

对于某些方法（如情感分析、文本分类），存在公开的标注数据集可以作为开发测试数据。例如，IMDB电影评论数据集、Twitter情感数据集、京东商品评论数据集等。使用公开数据集的优势在于，你可能同时获得"标准答案"（人工标注），便于直接计算准确率。劣势是数据集的主题可能与你的设计研究场景不匹配——如果你的Skill是分析金融产品的用户反馈，用电影评论训练或测试的模型可能无法迁移。

4.3.2 数据预处理的Vibe Coding实现

数据预处理是将原始数据转换为分析模块可以接收的结构化格式的过程。即使你使用的是大模型，它能够处理非结构化文本，预处理仍然有重要价值：它可以清洗噪声、统一格式、减少上下文噪音，从而提升分析的稳定性和质量。

常见预处理任务及Vibe Coding模板：

任务	Vibe Coding描述模板
文本分段	请写一个函数`segment_transcript(file_path)`，读取访谈文字稿txt文件，按自然段落分段。剔除完全空白的行、只包含时间戳标签的行、只有说话人标签没有内容的行。返回列表，每个元素为`{paragraph_id, text, char_count}`。如果一个段落超过500字，按句号进一步分割。
文本清洗	请写一个函数`clean_feedback(text_list)`，对每条文本：去除前后空白，统一全角/半角标点，将连续超过2个的重复字符缩减为2个，去除URL和纯emoji行。返回清洗后的列表和修改日志。
多文件加载	请写一个`DataLoader`类，支持从文件夹批量加载txt、docx、md文件。`load_all(folder_path)`返回列表，每个元素包含`filename`、`content`、`char_count`。跳过超过1MB的文件并记录警告。
问卷解析	请写一个函数`parse_survey_csv(csv_path)`，自动识别列类型：值域1-5且列名含"满意度"的标记为Likert；自由文本标记为开放题；固定选项标记为分类。返回各列的类型、示例值、缺失率。

预处理代码的快速检验方法：

在将预处理结果传给分析模块之前，建议做以下两项检验：

1. **眼检前10条**：打印出预处理后的前10条数据，肉眼判断格式是否整齐、关键信息是否丢失。
2. **统计对比**：让AI在代码中加入简单的统计输出："原始记录数X，预处理后条目数Y，平均字符长度Z。如果Y与X差异超过20%，输出警告。"

这些检验可以在几分钟内完成，但能避免因预处理错误导致分析结果全盘皆错。

4.4 核心分析逻辑：提示词设计与输出稳定性

4.4.1 分析函数的标准描述模板

在Vibe Coding中，分析函数的提示词应当遵循一个统一的结构化模板。这个模板确保AI能够生成输入输出明确、容错性强、易于对接的代码。

```
请写一个函数 `analyze_xxx(input_data)`。

【输入】input_data 是 [类型]，来自上一步 [预处理函数] 的输出。其结构为：{字段说明}。

【处理】调用 [模型名称] API，使用以下提示词框架进行分析：
  - 系统角色：你是一位[领域]分析师，擅长[能力描述]。
  - 分析框架：[列出必须分析的维度，如：功能模块、用户情绪、问题根因分类、设计启示]
  - 判断依据要求：请引述原文中的关键短语作为判断依据。
```

```
    - 不确定性处理：如果不确定，请在输出中标注 `confidence: low` 并给出不确定的原因。
    - 输出格式：必须为JSON，结构为 {字段列表}。不要添加任何额外解释。

【输出】函数返回一个字典，包含以下字段：
    - field1 (类型): 含义，取值范围
    - field2 (类型): 含义，取值范围
    ...

【解析】API返回的文本需要按以下方式解析：
    1. 首先尝试json.loads直接解析。
    2. 如果失败，使用正则提取 ```json ... ``` 代码块。
    3. 如果仍然失败，尝试修复常见格式错误（尾部多余逗号、单引号替换为双引号）。
    4. 所有解析失败则返回 `{"error": "解析失败", "raw": 原始文本}`。

【调用要求】使用指数退避重试（最多3次），API Key从环境变量读取，temperature=0.2，记录每次调用的耗时。
```

这个模板中的每一部分都有其存在的理由：

- **明确输入来源**：防止预处理输出与分析函数期望的类型不匹配。
- **分析框架显式化**：将你的方法知识直接编码进提示词，而不是让AI自己猜测。
- **要求判断依据**：提升可解释性，同时迫使模型更审慎。
- **多层次解析**：应对模型输出格式不稳定的常见问题。
- **非功能要求**：确保代码在生产环境中可靠运行。

4.4.2 分析提示词设计的五项原则

原则一：角色设定要具体但不过度。

✅ 好的示例："你是一位用户研究分析师，擅长从用户访谈中提取行为模式和潜在需求。"

❌ 不好的示例："你是一位拥有20年经验的用户体验专家，曾获多项行业大奖，对金融、医疗、教育等多个垂直领域有深入研究。"

原因：大模型对角色设定的第一句话最敏感，后续堆砌的描述不仅稀释信号，还可能触发模型去搜索训练数据中不存在的特定人物。

原则二：分析框架要在提示词中显式给出。

✅ 好的示例："请用以下框架分析每条反馈：1）所涉及的功能模块；2）用户的主要情绪；3）问题的根因分类（功能缺失/交互设计/性能/预期不符/其他）；4）对设计改进的启示（如有）。"

❌ 不好的示例："请分析每条反馈。"

设计研究的专业性体现在框架上。你提供给AI的框架越具体，AI的输出越接近一个训练有素的研究者的分析深度。

原则三：要求AI给出判断依据。

✅ 好的示例："对每条反馈，给出情感极性判断，并附上判断依据（引述原文中支持该判断的关键短语）。"

❌ 不好的示例："判断每条反馈的情感极性。"

要求依据有两个价值：第一，迫使模型更审慎；第二，让你能够快速验证判断是否合理。

原则四：为不确定性留出空间。

✅ 好的示例："如果你无法确定某个分类，请在输出中标注 `confidence: low` 并给出你的最佳猜测和不确定的原因。"

❌ 不好的示例："必须对每一条给出确定的分类。"

当模型被迫在不确定时也给出确定答案，它会"编造判断"。主动为不确定性留出空间，是负责任的设计研究AI工具应具备的能力。

原则五：输出格式指令要"冗余"。

在提示词的末尾再次重申格式要求，甚至给出一个格式示例。这种冗余在人类沟通中显得啰嗦，但在与模型沟通中显著提高格式遵从率。

示例：

```
请严格按照以下JSON格式输出，不要添加任何额外的解释文字：
{
  "sentiment": "正面/负面/中性",
  "reason": "判断依据",
  "confidence": "high/medium/low"
}
再次强调：只输出JSON，不要输出任何其他内容。
```

4.4.3 处理AI输出的不稳定性

即使采用上述设计原则，大模型API的输出仍然可能存在不稳定性——同样的输入在不同调用中可能得到不同的表述，甚至不同的判断。以下是在Vibe Coding层面管理不稳定性的方法。

方法一：降低温度参数。

温度控制模型的随机性——0是最确定性，1或更高则更随机。对于分析型任务，建议设置在0-0.3之间。

Vibe Coding描述：

> 在调用API时，将temperature参数设为0.1，确保分析结果尽可能稳定和可复现。

方法二：使用JSON模式。

如果API支持`response_format`参数，要求启用JSON模式。

Vibe Coding描述：

> 如果API支持response_format参数，请设置为{"type": "json_object"}，确保模型返回合法的JSON。

方法三：增加解析容错层。

Vibe Coding描述：

> 解析API返回的JSON时，请加入三层容错：1）直接json.loads；2）用正则提取`json...`块；3）修复尾部多余逗号、单引号等常见错误。

方法四：关键判断做一致性检查。

对于直接影响最终决策的关键判断（如"严重度≥4"），可以调用API两次，比较结果。

Vibe Coding描述：

> 对于severity≥4的关键判断，请调用API两次（使用相同的temperature），比较两次结果。如果两次判断不一致（一次判断为严重，一次不是），在输出中标注 `needs_review: true` 并同时展示两次判断结果。

4.4.4 当Vibe Coding卡住时：提示词调整策略

在实际开发中，你可能会遇到AI生成的代码运行但不正确、或反复报错无法解决的情况。以下是一些典型场景及提示词调整方向。

卡住场景	提示词调整方向	示例
分析结果区分度低（所有输出都一样）	增加分析深度要求	“当前分析结果区分度不够。请修改提示词，要求模型识别‘字面意思’和‘潜在含义’的区别。例如，‘还行吧’字面中性但可能表达不满。”
输出格式不稳定（JSON常常被包裹）	冗余强调格式 + 增强解析容错	“在提示词末尾加一行：‘记住：你的回复必须且只能是合法的JSON。不要添加任何解释。’同时在解析代码中增加提取 `json...` 块的逻辑。”
API频繁超时/限流	增加重试间隔和次数	“请增加重试策略：指数退避初始等待5秒，最大重试5次。每次重试前输出等待时间。”
框架代码过于复杂	要求简化实现	“当前使用的LangChain代码我难以理解。请改用纯Python和OpenAI库的简单实现，所有提示词用字符串定义，函数串联。”

一般原则：当卡住时，**增加约束而不是增加功能**。不要试图通过“再加一个功能”绕过当前问题——这通常会让代码更复杂、问题更难定位。你应该回到出问题的环节，在提示词中增加更精确的约束（输入输出格式、逻辑检查、错误处理等）。

4.5 交互流实现：从终端到Gradio

4.5.1 MVP阶段交互流的最低标准

在第4周的MVP阶段，交互流不需要华丽的界面或复杂的用户管理。最低标准是：**一个不是开发者本人也能操作的工具**。具体来说，满足以下三条即可：

1. **明确的输入方式**：用户知道在哪里提供数据（命令行参数、文件路径输入、文件上传组件等）。
2. **清晰的进度反馈**：处理过程中用户能看到当前在做什么（“正在提取帧...”“正在分析第3/20帧...”），而不是黑箱等待。
3. **有意义的输出**：最终产出的结果易于查看（终端打印摘要、保存文件、简单HTML等）。

4.5.2 终端交互式MVP的Vibe Coding模板

对于大多数Skill，终端交互已经足够满足MVP需求。以下是一个通用的描述模板：

```
请为上述处理链添加一个简单的命令行交互入口。用户运行脚本时：

1. 如果提供了命令行参数（如 `python main.py data/test.txt`），使用该文件路径作为输入。
2. 如果没有提供参数，提示用户输入文件路径：`请输入数据文件路径：`
3. 如果文件不存在或格式不支持，给出明确的错误提示并退出。
4. 处理过程中显示阶段性进度：
   - `[1/3] 正在加载数据...`
   - `[2/3] 正在分析数据（已处理 5/20 条）...`
   - `[3/3] 正在生成报告...`
5. 处理完成后，在终端打印分析结果摘要（如“共分析20条，检测到5个可用性问题，严重度≥4的有2个”），并告知用户完整结果保存的路径（如“报告已保存到 report.md”）。
6. 询问用户是否要继续分析另一个文件（输入 y/n）。
```

4.5.3 Gradio轻量界面MVP的Vibe Coding模板

如果你在第3周已经搭建了Gradio原型骨架，可以在其基础上增加真实的处理逻辑。以下是一个描述模板：

```
请在我的Gradio界面上增加以下功能（基于已有的骨架代码）：

1. 文件上传组件：支持上传txt或csv文件，上传后显示文件名和大小。
2. 一个“开始分析”按钮。点击后：
   - 按钮文字变为“分析中...”，并禁用按钮
   - 读取上传文件的内容
   - 依次调用预处理函数和分析函数
   - 每完成10%的处理进度，更新一个进度条组件
   - 分析完成后，按钮恢复为“开始分析”，进度条归零
3. 结果展示区域：使用Gradio的Markdown或JSON组件展示分析结果摘要。
4. 一个“下载结果”按钮：将完整的分析结果（JSON格式）保存为文件供用户下载。

界面布局：左侧是上传区和控制区，右侧是结果展示区。
```

4.5.4 连接预处理与分析：胶水函数的重要性

在Vibe Coding中，一个常见的陷阱是：预处理模块和分析模块由不同时间、不同提示词生成，导致它们之间的数据结构不匹配——预处理输出一个字典列表，但分析函数期望一个字符串列表。这种不匹配在运行时会引发难以追踪的错误。

解决方案：**让AI生成一个“胶水函数”作为主入口**，显式地连接各个模块。

Vibe Coding描述：

```
请写一个 `run_pipeline(input_source)` 函数作为整个处理链的入口。它依次调用：

1. `data = load_and_preprocess(input_source)`  （从预处理模块导入）
2. `results = analyze_batch(data)`              （从分析模块导入）
3. `report = generate_report(results)`          （从报告模块导入）

在每一步之间，加入类型和内容检查：
- 检查 `data` 是否是非空列表，如果不是，抛出异常“预处理返回空数据”
- 检查 `results` 中每个元素是否包含必需的字段（has_issue, severity等），如果缺失，记录警告并填充默认值
- 如果任何检查失败，保存当前状态快照到 `debug_snapshot.json` 用于调试
```

这个胶水函数的存在，使得你可以独立测试每个模块，同时在集成时获得清晰的错误信息。

4.6 MVP内部检视：展示、分享与互助

4.6.1 检视的目的与规则

在每周开发周期的最后，进行MVP内部检视是课程的重要环节。检视不是考核评分，而是三重的互助活动：

1. **进度校准**：看到其他组的成果，你可以判断自己的进度是落后、超前还是大致同步。
2. **经验流动**：每个组都会积累一些独特的Vibe Coding技巧或踩坑经验，检视让这些经验在全班范围内共享。
3. **集中排障**：如果某个组遇到无法解决的问题，全班可以一起提供思路。

检视展示格式（每组7分钟）：

- 3分钟：展示MVP（现场运行或演示关键代码+运行结果）
- 2分钟：分享一个“本周最有效的提示词”
- 1分钟：分享一个“本周最坑的报错”及解决方式
- 1分钟：提出一个当前仍卡住的问题（如果有），向全班求助

4.6.2 MVP的及格线与加分项

为了帮助你在准备检视时有明确的目标，以下是明确的评价标准：

及格线（必须达到）：

- 一个组外的用户（或另一组的同学），在你的电脑上，按照你的口头指引，能够提供一个测试输入并触发处理流程，得到一个有意义的输出。

加分项（展示进阶能力）：

- 处理链中包含了至少一个人工确认环节（展示交互式架构的雏形）
- 输出格式清晰、结构化，而不仅仅是一段文本
- Vibe Coding对话日志记录详实、有组织
- 端到端已经跑了至少一轮完整的数据（而非只处理了一条测试数据）

不在及格线上的状态：

- 代码写好了但从没跑通过（哪怕一次）
- 数据完全没准备，演示时只能展示空代码
- 分析函数是mock的假分析，没有调用真实AI模型

4.6.3 跨组学习：听什么、问什么

在观看其他组展示时，建议你重点听取以下四个方面：

1. **提示词设计**：他们的分析提示词是怎么写的？有没有什么结构或措辞可以借鉴？
2. **报错故事**：有没有你还没遇到过但很可能未来会遇到的报错？他们的解决方案是什么？
3. **架构选择**：他们选择了管道式还是交互式？人工节点放在哪里？为什么？
4. **求助问题**：如果某组提出一个卡住的问题，你有没有相关经验可以分享？

在提问环节，推荐使用“具体而开放”的问题形式：“我看到你们在分析提示词中使用了XXXX，为什么选择这种表述？你们有没有尝试过其他方式？”

4.7 本章小结

本章是整个课程中从“设计”跨越到“实现”的关键一章。我们通过一个完整的Vibe Coding示范——可用性测试高亮时刻提取器——展示了从零到可运行MVP的三轮推进过程：数据准备与预处理、核心AI分析、交互流与集成。每一轮都有明确的产出、可检验的标准以及与后续模块的接口。

在此基础上，我们提供了数据准备指南（真实数据脱敏、AI生成模拟数据、预处理Vibe Coding）、核心分析逻辑开发指南（提示词设计原则、输出稳定性处理、常见卡点应对）、以及交互流实现指南（终端和Gradio模板、胶水函数）。这些指南构成了一个可操作的工具箱，无论你选择的设计研究方法是什么，都可以直接应用。

最后，通过MVP内部检视的规则和标准，我们明确了本章结束时应该达到的成果底线，以及如何通过跨组学习加速自己的开发进程。

第4章的完成意味着你的Skill已经从“蓝图”变成了“第一块砖石”。在第5章中，我们将在MVP的基础上进行迭代优化，引入用户反馈，提升分析准确率，并让交互体验从“能用”走向“好用”。

思考与练习

一、概念理解题

1. 什么是Vibe Coding开发中的“分轮推进”原则？为什么它比“一个提示词生成整个系统”更有效？
2. 在分析提示词设计中，为什么要显式给出“分析框架”而非仅仅告诉模型“请分析”？请结合你自己所选的方法举例说明。
3. 数据预处理在MVP开发中扮演什么角色？如果一个Skill跳过了数据预处理，直接将原始文本喂给大模型，可能产生哪些问题？
4. 什么是“胶水函数”？它在连接预处理模块和分析模块时起到什么作用？

二、分析应用题

5. 以下是一个分析提示词的设计，请指出其中至少三个可以改进的地方，并写出改进后的版本。

   > “请分析以下用户反馈的情感。输出情感极性。”

6. 假设你的Skill在运行时报错：`KeyError: 'sentiment'`，而你知道分析函数本应返回一个包含`sentiment`字段的字典。请设计一个Vibe Coding对话，向AI描述这个问题并请求修复。写出你会使用的提示词。

三、实践与反思题

7. 以小组为单位，完成本章4.2节示范案例的复现（可用性测试高亮时刻提取器）。使用任意一段可用性测试视频（或模拟视频），记录你的三轮提示词、AI生成的代码、运行结果和遇到的调试过程。提交一份Vibe Coding对话日志。
8. 基于你自己的Skill选题，按照本章4.3-4.5节的指南，完成以下工作：
   - 准备15-30条开发数据（真实或模拟）
   - 编写数据预处理代码（Vibe Coding生成）
   - 编写核心分析函数（Vibe Coding生成）
   - 实现终端或Gradio交互流
   - 运行完整的端到端流程，记录输出结果

   将上述所有代码和运行截图整理为一份MVP开发报告。
9. **个人反思**（500字）：在MVP开发过程中，你遇到的最大技术障碍是什么？你是如何通过与AI对话解决的？你在“向AI描述问题”这件事上学到了什么新的经验？

参考文献

[1] Ries, E. (2011). *The Lean Startup*. Crown Business.

[2] Karpathy, A. (2024). Vibe coding: A new way to program. *Personal blog / X*.

[3] OpenAI. (2024). GPT-4V(ision) system card. *OpenAI Research*.

[4] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.

[5] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. *OSDI*.

[6] Chen, M., et al. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.